



UNIVERSITÀ DEGLI STUDI DI SALERNO

Dipartimento di Informatica
Corso di Laurea Triennale in Informatica

TESI DI LAUREA

Integrazione del plugin cASpER per la rilevazione automatica di code smell nell'ambito della Continuous Integration

RELATORE

Prof. **Andrea De Lucia**

Università degli studi di Salerno

CANDIDATO

Antonio Trovato

Matricola: 0512105754

Anno Accademico 2020-2021

Abstract

Nell'ambito dell'ingegneria del software, il cambiamento è la regola, piuttosto che l'eccezione. La prima legge di Lehman sull'evoluzione del software ci insegna che un programma deve essere costantemente modificato per rimanere utile; la seconda, che un programma diventa sempre più complesso, man mano che si modifica, a meno che non si effettui uno sforzo per ridurre tale complessità. Così come i progetti software si fanno sempre più complessi, anche la loro produzione e gestione si fa sempre più difficile. In particolare, soprattutto nel contesto di progetti che richiedono l'impiego di diversi sviluppatori o team di sviluppo, alcuni problemi che frequentemente si riscontrano riguardano la gestione di cambiamenti continui e refactoring delle componenti implementate, nonché l'integrazione delle componenti sviluppate dai vari team durante le fasi di sviluppo. Tuttavia, gli sviluppatori dispongono di numerosi strumenti volti ad affrontare al meglio queste problematiche, tra questi figurano cASpER, e la piattaforma Travis CI. cASpER è un plugin di IntelliJ IDEA che fornisce un supporto grafico e semi-automatico per l'individuazione ed il refactoring dei cosiddetti code smell. Molto spesso, infatti, soprattutto di fronte a risorse limitate o a scadenze stringenti, si è spesso costretti a trascurare le buone pratiche di progettazione e programmazione. Tutto ciò implica l'introduzione di un cosiddetto debito tecnico, che cASpER aiuta a "ripagare". Travis CI, definito dall'omonima compagnia, è invece una piattaforma il cui compito è quello di implementare e fornire il servizio detto di "Continuous Integration", la pratica di effettuare frequenti sincronizzazioni dei lavori dei singoli sviluppatori e dei team del progetto, anche per ogni lieve modifica; il vantaggio consiste nel fatto che si dovrà spendere molto meno tempo per la correzione ed il rilevamento di problemi dovuti all'integrazione dei lavori prodotti. Nell'ambito di questa tesi il plugin cASpER è stato esteso per essere integrato in un contesto di Continuous Integration. Viene dunque proposta una nuova versione di cASpER che prevede non solo l'indipendenza del plugin dal suo "ambiente" originario (ossia IntelliJ IDEA), ma che connette anche le sue funzionalità ai servizi messi a disposizione da Travis CI. Dunque, ogni volta che uno sviluppatore effettuerà un commit indirizzato alla repository del progetto al quale sta lavorando, cASpER sarà eseguito per individuare eventuali code smell presenti nelle componenti da integrare. Spetterà, poi, allo sviluppatore decidere se correggere gli eventuali smell individuati, oppure procedere con l'integrazione.

Indice

1	Introduzione	4
1.1	Contesto applicativo	4
1.2	Motivazioni e Obiettivi	6
1.3	Risultati	7
1.4	Struttura della tesi	7
2	Stato dell'arte su Continuous Integration ed uso di Travis CI	9
2.1	Introduzione	9
2.2	Uso della Continuous Integration in Ingegneria del Software . . .	10
2.2.1	L'utilità della Continuous Integration	10
2.2.2	I benefici della Continuous Integration	10
2.2.3	Svantaggi della CI	11
2.3	Uso di Travis CI	12
3	Stato dell'arte sul plugin cASpER	13
3.1	Introduzione	13
3.2	Il funzionamento di DECOR	14
3.2.1	L'identificazione dei code smell tramite l'analisi strutturale	14
3.2.2	La descrizione di DETEX	16
3.3	Il funzionamento di TACO	18
3.3.1	Individuazione dei code smell tramite l'analisi testuale . .	18
3.3.2	I Code Smell e la loro identificazione	19
3.4	Funzionamento di cASpER per IntelliJ IDEA	22
3.4.1	Le funzionalità di cASpER	23
3.4.2	Architettura di cASpER	25
3.4.3	Esempio di utilizzo di cASpER	25
4	cASpER per la Continuous Integration	28
4.1	Componenti Riutilizzate e Rimosse	29
4.2	Nuove Componenti Implementate	32
4.3	Integrazione in Travis CI	36
5	Un esempio di utilizzo	37
5.1	Rilevamento di code smell	37

6	Conclusioni e sviluppi futuri	40
6.1	Conclusioni	40
6.2	Sviluppi Futuri	41

Capitolo 1

Introduzione

1.1 Contesto applicativo

Il processo di realizzazione di un software richiede una fase di studio e progettazione che risulta propedeutica a quella di programmazione. Più nello specifico, lo sviluppo di un software comincia sempre con un'analisi dei requisiti nella quale vengono rilevati tutti gli aspetti più importanti che il prodotto finale dovrà rispettare; si procede successivamente a una fase di progettazione o design che ha lo scopo di definire l'architettura e le strategie per l'implementazione vera e propria del software. L'implementazione stessa non è mera scrittura di codice ma di pari passo essa prevede una fase di testing volta a dimostrarne la validità.

Naturalmente però non esiste alcun software che una volta fornito al cliente non necessiti di manutenzione e rinnovi periodici; ecco perché il ciclo di vita di un sistema software va ben oltre il suo sviluppo. Ciò è messo in evidenza dalla prima legge di Lehman sull'evoluzione del software:

Cambiamento continuo: Un sistema usato in ambienti reali necessariamente deve cambiare o diventare progressivamente meno utile in quell'ambiente.

Un software dunque necessita di una manutenzione che spesso può risultare complessa da eseguire a causa di due principali motivazioni:

- la prima è legata alla mancanza di organizzazione durante la fase di progettazione;
- la seconda riguarda aspetti tecnici, quali ad esempio la difficoltà nel comprendere il codice scritto da altri o la scarsa predisposizione del software stesso ad eventuali modifiche.

Al fine di rendere il software più facilmente manutenibile, bisognerebbe salvaguardare il suo livello di qualità seguendo in particolare le linee guida del

paradigma di programmazione Object Oriented[1] (su cui si basano i nostri studi).

I parametri da tenere costantemente presenti nell'implementazione di software Object Oriented sono principalmente: coesione ed accoppiamento.

La coesione misura il grado di interdipendenza tra le componenti (metodi) di una classe. Poiché ad ogni classe dovrebbe essere attribuita una singola responsabilità, si rende desiderabile la presenza di un alto grado di coesione.

L'accoppiamento invece misura il grado di interdipendenza fra le classi; pertanto minore risulta il grado di accoppiamento, meno la modifica di una classe impatterà sulle altre classi.

Dunque un software è facilmente manutenibile quando il grado di coesione è alto mentre quello di accoppiamento è basso.

Tuttavia tale scenario è difficilmente raggiungibile in quanto, per aumentare la coesione occorre ampliare il numero di classi in maniera che ognuna di esse implementi sempre meno responsabilità. Ciò, però, rischia di accrescere l'interdipendenza tra le classi, che si traduce in un aumento dell'accoppiamento. Per questo motivo una buona scomposizione consiste nel trovare un giusto equilibrio tra coesione ed accoppiamento.

La manutenzione risulta poco accurata laddove i parametri di coesione ed accoppiamento vengono sottovalutati. In questo caso torna utile la seconda legge di Lehman.

Entropia crescente: Quando un sistema cambia, la sua struttura tende a diventare più complessa. Risorse extra devono essere utilizzate per preservare e semplificare la sua struttura.

La procedura denominata *Refactoring* si riferisce per l'appunto all'insieme delle periodiche operazioni di semplificazione e miglioramento del software.

Un'altra pratica oramai consolidata nell'ambito dell'Ingegneria del Software è quella della Continuous Integration che consiste nella frequente sincronizzazione ed unione dei lavori degli sviluppatori dei team del progetto, anche per lievi aggiornamenti, al fine di semplificare il processo di integrazione dei prodotti ottenuti.

1.2 Motivazioni e Obiettivi

Le difficoltà maggiori incontrate dagli sviluppatori derivano da diversi fattori quali:

- l'esigenza di implementare del codice di buona qualità che garantisca un'efficace manutenzione;
- richieste di aggiornamenti continui da parte dei clienti;
- scadenze di consegna spesso incalzanti.

Pertanto gli sviluppatori sono costretti a scegliere tra una buona programmazione, passando cioè attraverso le canoniche fasi di progettazione ed analisi, e la diretta implementazione delle soluzioni richieste. Proprio a causa di mancanza di tempo o documentazione esauriente, la seconda opzione è spesso quella ritenuta migliore, tuttavia è anche quella che vede sacrificata la buona qualità del software e che di conseguenza produce dei difetti tecnici piuttosto rilevanti[2]. Questi ultimi prendono il nome di *bad code smell* (oppure *code smell* o ancora *smell*), che appunto rappresentano scelte non ottimali di progettazione ed implementazione da parte dei programmatori nello sviluppo del software.

Nell'ambito dell'Ingegneria del Software, sono molti gli studi che i ricercatori hanno condotto proprio sui *code smell* evidenziando non solo la loro rilevanza ed il loro impatto sulle singole versioni del software, ma anche dimostrandone la longevità e dunque il loro effetto nelle versioni successive del software stesso[16].

Si è visto come i difetti di progettazione, oltre a rendere complessa la comprensione del codice, influiscano negativamente su altri requisiti non funzionali, come ad esempio la manutenibilità, di un sistema software. Proprio sulla base di tali studi sono stati generati diversi tool per la rilevazione e, ove possibile, la correzione automatica di *smell* tramite l'applicazione di particolari strategie di analisi del codice sorgente. Nello specifico, le strategie che più delle altre si sono mostrate molto valide sono quelle strutturali e quelle testuali, implementate da due diversi approcci di rilevamento dei *code smell* che sono rispettivamente: DECOR (DEtection and CORrection) e TACO (Textual Analysis for Code smell detectiOn).

TACO e DECOR sono in grado di riconoscere quattro tipi di *code smell*: *Blob*, *Feature Envy*, *Misplaced Class*, *Promiscuous Package*. Proprio tramite l'integrazione di TACO e DECOR è stato implementato cASpER[15], un plugin per IntelliJ IDEA che ha lo scopo di automatizzare i processi di rilevamento e correzione di *code smell*.

Il presente lavoro è finalizzato per l'appunto all'espansione del plugin cASpER, che lo renda indipendente da IntelliJ IDEA. Si intende altresì includere nell'ambito della Continuous Integration sulla piattaforma Travis CI la nuova versione di cASpER ma per il solo rilevamento di *code smell*. Tale integrazione permetterebbe di eseguire cASpER, per ogni nuova modifica apportata al codice del sistema che si sta sviluppando, sull'intero codice del progetto come se fosse un test, permettendo agli sviluppatori di mantenere il codice del progetto il più esente possibile da code smell.

1.3 Risultati

Come ci si era prefissati, il plugin cASpER è stato reso totalmente indipendente da IntelliJ IDEA tramite l'utilizzo di un nuovo parser già precedentemente sviluppato. A tale scopo sono state prodotte da zero delle componenti per l'integrazione del nuovo parser. Inoltre, affinché si possa usufruire di cASpER tramite Travis CI, il plugin è stato esteso e gli è stata conferita la possibilità di essere eseguito da linea di comando.

La scelta di Travis CI è stata dovuta alla semplicità dell'integrazione con GitHub che è la piattaforma di pubblicazione di progetti software più utilizzata al mondo.

1.4 Struttura della tesi

Nei prossimi capitoli si descriveranno dettagliatamente tutti gli argomenti cui si è fatto cenno. Questo lavoro di tesi si compone di cinque capitoli organizzati come segue:

- **Capitolo 2. Stato dell'arte su Continuous Integration ed uso di Travis CI:** Il capitolo illustra il concetto di Continuous Integration e più in particolare lo stato dell'arte riguardo al servizio Travis CI che è stato utilizzato per poter raggiungere l'obiettivo di questo lavoro.
- **Capitolo 3. Stato dell'arte sul plugin cASpER:** Il capitolo illustra il funzionamento della versione di cASpER sulla quale si è lavorato per poter raggiungere lo scopo del lavoro di tesi. Inoltre individua le principali tecniche per il rilevamento e la correzione di code smell.

- **Capitolo 4. cASpER per la Continuous Integration** Questo capitolo rappresenta il cuore del lavoro di tesi svolto, in quanto descrive con precisione tutti i passaggi che sono stati effettuati per il suo compimento.
- **Capitolo 5. Un esempio di utilizzo** Questo capitolo mostra semplicemente un esempio di utilizzo del lavoro sviluppato.
- **Capitolo 6. Conclusioni e sviluppi futuri** Questo capitolo riassume in breve sia gli obiettivi che il presente lavoro si propone di risolvere, sia come tali obiettivi sono stati raggiunti. Inoltre indica delle possibili evoluzioni che il lavoro svolto potrebbe subire.

Capitolo 2

Stato dell'arte su Continuous Integration ed uso di Travis CI

2.1 Introduzione

Una delle problematiche più frequenti nell'ambito dell'Ingegneria del Software consiste nell'integrazione dei lavori effettuati dagli sviluppatori o dai team che lavorano ad un progetto. Infatti durante la fase di integrazione ci si trova molto spesso a dover affrontare non poche difficoltà dovute alla sincronizzazione di codice prodotto da programmatori diversi; appare pertanto piuttosto utile un meccanismo che automatizzi correttamente tale sincronizzazione.

La Continuous Integration è la pratica che consiste appunto nell'automatizzare l'integrazione di cambiamenti o aggiornamenti che più sviluppatori operano su un progetto software. I programmatori in pratica hanno la possibilità di sincronizzare i loro lavori ogni volta che venga effettuata una modifica o che vengano ampliate le caratteristiche del sistema in via di sviluppo.

Il grande vantaggio della continuous integration è che l'integrazione viene effettuata fin da subito, non si attende che ogni team abbia terminato i propri compiti. E' evidente, altresì, che il codice caricato nella repository centrale quotidianamente dagli sviluppatori è in genere di piccole dimensioni e senza dubbio molto inferiore rispetto alla quantità di materiale che essi dovrebbero caricare se volesero integrare direttamente gli interi artefatti ottenuti completando i propri task. In questo modo la Continuous Integration consente l'integrazione di piccole porzioni di codice alla volta, snellendo e dunque velocizzando di molto le sincronizzazioni.

Va da sè che con tale procedimento il processo di sviluppo sia gestito in maniera molto più agile e controllato e il prodotto finale risulti molto più flessibile ad eventuali modifiche; questo significa che la manutenibilità del software che si sta realizzando si rivela migliore.

Ai fini di questa tesi l'utilizzo della Continuous Integration è stato possibile grazie all'utilizzo del servizio Travis CI.

2.2 Uso della Continuous Integration in Ingegneria del Software

2.2.1 L'utilità della Continuous Integration

Si può verificare l'importanza della Continuous Integration[17] solo qualora ci si soffermi sulle difficoltà che si possono incontrare quando tale pratica non venga osservata. Infatti, senza di essa gli sviluppatori sono costretti a coordinarsi comunicandosi esplicitamente la volontà di contribuire al codice.

Quindi in un ambiente nel quale non sia presente l'uso della Continuous Integration si verifica che i costi generali di realizzazione del progetto aumentino anche vertiginosamente. Questo perché le comunicazioni tra i team sono complesse da gestire e costringono inevitabilmente gli sviluppatori non solo a dover attendere risposte dagli altri colleghi, quindi ad allungare il tempo di realizzazione del prodotto, ma comportano anche un incremento della frequenza degli errori commessi durante lo sviluppo. Inoltre è chiaro che man mano che il codice aumenta, maggiori sono le integrazioni che dovranno essere effettuate.

Nel momento in cui viene introdotta la pratica della Continuous Integration all'interno di un progetto, gli sviluppatori sono finalmente in grado di lavorare contemporaneamente su funzionalità diverse del software e nel momento in cui sono pronti ad integrare il proprio lavoro con quello degli altri esso può essere eseguito rapidamente e soprattutto automaticamente.

2.2.2 I benefici della Continuous Integration

I vantaggi nell'uso della Continuous Integration[17] sono molteplici; tra i tanti ci basta considerare che la rapidità con la quale tale pratica ci consente di rilevare eventuali errori permette anche una loro celere risoluzione, il che implica un aumento nella produzione di release. Inoltre, appare evidente come la facilità di uso della Continuous Integration permetta anche frequenti aggiornamenti senza costi elevati, consentendo al software di mantenersi sempre aggiornato e dunque aumentando costantemente la qualità del prodotto finale.

Applicando la Continuous Integration aumenta anche l'affidabilità dei test, in quanto, dovendo essere integrate piccole porzioni di codice, queste ultime consentono di effettuare controlli più mirati che permettono di conseguire risultati più sicuri e precisi. Questo significa che anche i problemi cosiddetti "non-critici" vengano identificati con largo anticipo, risultando in release più pulite e corrette.

L'acronimo MTTR (Mean Time To Resolution)[18] indica la misura di manutenibilità e riparabilità di un software e, in genere, una buona applicazione della Continuous Integration implica un'importante riduzione di tale misura. Ciò è dovuto sempre alla relativa snellezza del codice da integrare che consente di identificarne molto più rapidamente i difetti.

Tutti i miglioramenti che si ottengono dalla Continuous Integration non hanno solo effetto sullo sviluppo del software, ottimizzando la qualità del software che si sta sviluppando, ma permettono di conseguire considerevoli progressi anche riguardo all'intera organizzazione di un progetto[19]; la comunicazione tra i team sarà infatti molto più veloce e facile da gestire[19]. Inoltre l'ottimalità del software implicherà maggiore soddisfazione da parte dei clienti e la possibilità di studiare strategie di ingresso nel mercato molto più precise[19].

2.2.3 Svantaggi della CI

Come per ogni altra tecnologia, anche la Continuous Integration presenta qualche difetto[17]. In particolare se ne possono identificare due.

Il primo consiste nell'esigenza di affidarsi ad un servizio aggiuntivo al fine di poter includere nei propri progetti la Continuous Integration; ciò generalmente implica l'utilizzo di qualche risorsa in più. Per esempio ai fini di questa stessa tesi è stato utilizzato un servizio di Continuous Integration detto Travis CI.

Il secondo difetto riguarda invece la possibilità che si creino code nel caso in cui più sviluppatori richiedano contemporaneamente l'integrazione dei propri lavori.

In generale si è però visto che i vantaggi dell'applicazione della Continuous Integration superano di gran lunga gli svantaggi; ecco perché tale pratica sta diventando sempre più importante nell'Ingegneria del Software.

2.3 Uso di Travis CI

Uno dei principali servizi per l'applicazione della Continuous Integration nei propri progetti software è Travis CI[20], una piattaforma gratuita molto popolare che può essere facilmente connessa a GitHub, il provider di spazio hosting per progetti software più utilizzato nel mondo.

Integrandosi con GitHub, ogni qualvolta che uno sviluppatore effettua il caricamento di porzioni di codice nella repository del progetto, il compito di Travis CI è quello di lanciare, dopo aver eseguito le operazioni di build del software ed in maniera completamente automatica, una sequenza di test su tali porzioni di codice.

Al fine di attivare le build ed i test, andrà configurato all'interno della repository centrale del progetto un documento denominato ".travis.yml", il quale indica a Travis CI i compiti da eseguire. Se il codice caricato supererà tutti i test lanciati da Travis CI allora si potrà affermare che esso è completamente e correttamente integrato con il resto del progetto.

Uno dei motivi del successo di Travis CI consiste nell'ampio ventaglio di configurazioni che sono supportate, mostrando una grande adattabilità e flessibilità. Infatti, Travis CI non solo può essere configurato su un gran numero macchine con diversi sistemi operativi ma inoltre supporta software scritti nei più svariati linguaggi di programmazione.

Solo per fare qualche menzione, alcuni dei linguaggi supportati da Travis CI sono: C, C++, Java, PHP, Ruby, Go, Python, R, Swift, JavaScript, Visual Basic e molti altri.

Capitolo 3

Stato dell'arte sul plugin cASpER

3.1 Introduzione

Come si è visto sia l'uso della Continuous Integration che la rilevazione e correzione dei *code smell* sono diventate delle pratiche fondamentali nell'ambito dell'Ingegneria del Software.

Gli studi che si sono svolti negli ultimi anni in merito ai *code smell*, hanno rivolto particolari attenzioni allo sviluppo di metodologie per l'identificazione automatica dei difetti del codice sorgente. In particolare, tali tecniche di identificazione si basano sulle definizioni dei difetti della progettazione del codice e delle euristiche per il loro rilevamento che sono esposte in quattro libri fondamentali.

Il primo, di Webster[21], si sofferma sui rischi dello sviluppo di codice Object-Oriented, descrivendo inoltre le modalità di individuazione, o di prevenzione, dei problemi che potrebbero sorgere nelle fasi del ciclo di sviluppo del software.

Riel nel secondo libro[22] traccia più di sessanta euristiche per il controllo dell'integrità di un software. Chiaramente non si tratta di leggi infrangibili e dunque ci si può eventualmente permettere di trascurarne qualcuna.

Martin Fowler nel terzo libro[1], concentrandosi solo sugli *smell* strettamente connessi allo sviluppo software, descrive ben 22 tipi di *code smell* con le relative tecniche di refactoring volte a risolverli.

Brown et al. espongono nel quarto libro[23] 40 *smell* classificandoli successivamente nelle seguenti classi: *smell* di sviluppo, *smell* architetturali e *smell*

manageriali. Riguardo gli smell di sviluppo vengono anche fornite euristiche per la loro identificazione.

Sulla base delle informazioni contenute all'interno dei quattro libri appena citati sono stati successivamente sviluppati diversi strumenti per il rilevamento ed eventualmente per la correzione dei *code smell*.

Ai fini di questa tesi sono stati utilizzati due strumenti fondamentali: Travis CI per la Continuous Integration ed il plugin cASpER, un software sviluppato per IntelliJ IDEA il cui scopo consiste nel rilevamento e nella correzione semiautomatiche di *code smell*.

Il plugin cASpER per IntelliJ IDEA è uno strumento semiautomatico per il rilevamento e la correzione di difetti nella progettazione del codice detti *bad code smell*; tale rilevamento si basa su due approcci già conosciuti: DECOR[3], che sfrutta un'analisi strutturale del codice, e TACO[2], che invece utilizza un'analisi testuale del codice.

In generale il plugin è in grado di:

- rilevare *smell* usando gli approcci DECOR e TACO;
- proporre delle operazioni di refactoring che implementano approcci già ampiamente proposti in letteratura;
- modificare in maniera automatica il codice sorgente affetto da *smell* seguendo le specifiche del refactoring scelto;
- mostrare importanti informazioni del codice sorgente quali: metriche, concetti ed attributi.

Nelle prossime sezioni si analizzeranno l'approccio utilizzato da DECOR e l'approccio utilizzato da TACO per il rilevamento di smell. Successivamente si parlerà del calcolo probabilistico degli smell.

3.2 Il funzionamento di DECOR

DECOR è un noto metodo il cui scopo consiste nella realizzazione di tecniche di rilevamento di code smell. In particolare DECOR descrive esplicitamente una serie di passaggi da seguire per la realizzazione di tali tecniche.

3.2.1 L'identificazione dei code smell tramite l'analisi strutturale

I passaggi descritti da DECOR sono i seguenti:

- **Step 1. Description Analysis:** tramite la descrizione testuale esistente in letteratura, estrapoliamo i concetti chiave dei vari tipi di code smell esistenti creando in questo modo un vero e proprio vocabolario riutilizzabile e che ne descriva le caratteristiche;
- **Step 2. Specification:** i concetti che sono presenti nel vocabolario vengono combinati per specificare gli smell in maniera consistente;
- **Step 3. Processing:** le specifiche ottenute sono tradotte in algoritmi da applicare per il rilevamento;
- **Step 4. Detection:** il rilevamento è effettuato sul sistema software che si sta sviluppando applicando gli algoritmi ottenuti nel passaggio precedente, estraendo in questo modo la lista di componenti che si sospetta costituiscano uno smell.
- **Step 5. Validation:** gli smell sospetti vengono via via validati al fine di confermare che si tratti di reali smell.

Il primo step, in quanto generico, va applicato su un effettivo set di smell. Il secondo ed il terzo step vanno eseguiti quando ci si trova di fronte alla necessità di dover definire nuovi smell. Gli ultimi due vanno sempre applicati e per qualsiasi sistema.

Una corretta esecuzione di DECOR prevede un approccio iterativo.

Pertanto si procede come segue: si effettua il primo step al fine di estendere il vocabolario di code smell. Tramite il secondo ed il terzo step effettuiamo, qualora fosse necessario, rispettivamente un'estensione del linguaggio della specifica ed un raffinamento, nonché una ripetizione della specifica per ridurre il numero di rilevamenti errati. I criteri di terminazione sono scelti dagli sviluppatori i quali potrebbero avere particolari esigenze. Infine, dopo l'iterazione si conclude con gli ultimi due step.


```

1 RULECARD: Blob {
2   RULE: Blob
3     {ASSOC: associated FROM: mainClass ONE TO: DataClass MANY};
4   RULE: mainClass
5     {UNION LargeClassLowCohesion ControllerClass};
6   RULE: LargeClassLowCohesion
7     {UNION LargeClass LowCohesion};
8   RULE: LargeClass
9     {(METRIC: NMD + NAD, VERY-HIGH, 20)};
10  RULE: LowCohesion
11    {(METRIC: LCOMB, VERY-HIGH, 20)};
12  RULE: ControllerClass
13    {UNION (SEMANTIC: METHODNAME, {Process, Control, Command, Manage, Drive, System}),
14      (SEMANTIC: CLASSNAME, {Process, Control, Command, Manage, Drive, System})};
15  RULE: DataClass
16    {(STRUCT: METHODACCESSOR, 90)};

```

Figura 3.1: Rule Card per l'identificazione di un Blob

3.2.2 La descrizione di DETEX

In particolare cASpER si basa su un'istanza del metodo DECOR, chiamata DETEX. Quest'ultimo è composto da quattro passaggi che rappresentano delle effettive implementazioni dei corrispondenti passaggi di DECOR.

Ecco gli step che compongono DETEX:

- **Step 1. Domain Analysis:** il primo step effettua una completa analisi del dominio relativo agli smell al fine di identificare i loro concetti chiave tramite le loro descrizioni testuali. Grazie ai concetti ottenuti, oltre ad un vero e proprio vocabolario, si è in grado di ottenere una vera e propria tassonomia e classificazione degli smell. La tassonomia permette di sottolineare le somiglianze e le differenze tra gli smell identificati;
- **Step 2. Specification:** la specifica è eseguita usando un linguaggio domain-specific (DSL) che genera *rule card* (Figura 3.1) tramite vocabolario e tassonomia precedentemente ottenuti. Una rule card è un insieme di regole, ossia proprietà che una classe deve avere per essere considerata come smell. Quindi il DSL non solo definisce tali proprietà ma specifica anche le relazioni strutturali tra esse ed altre proprietà caratteristiche degli smell, quali le proprietà lessicali e strutturali e i relativi attributi interni;
- **Step 3. Algorithm Generation:** gli algoritmi di rilevamento vengono generati in maniera del tutto automatica a partire dai modelli estrapolati dalle rule card. Tali modelli sono ottenuti istanziando le regole delle card tramite parser dedicati;

- **Step 4. Detection:** finalmente vengono applicati in maniera completamente automatica gli algoritmi di rilevamento.

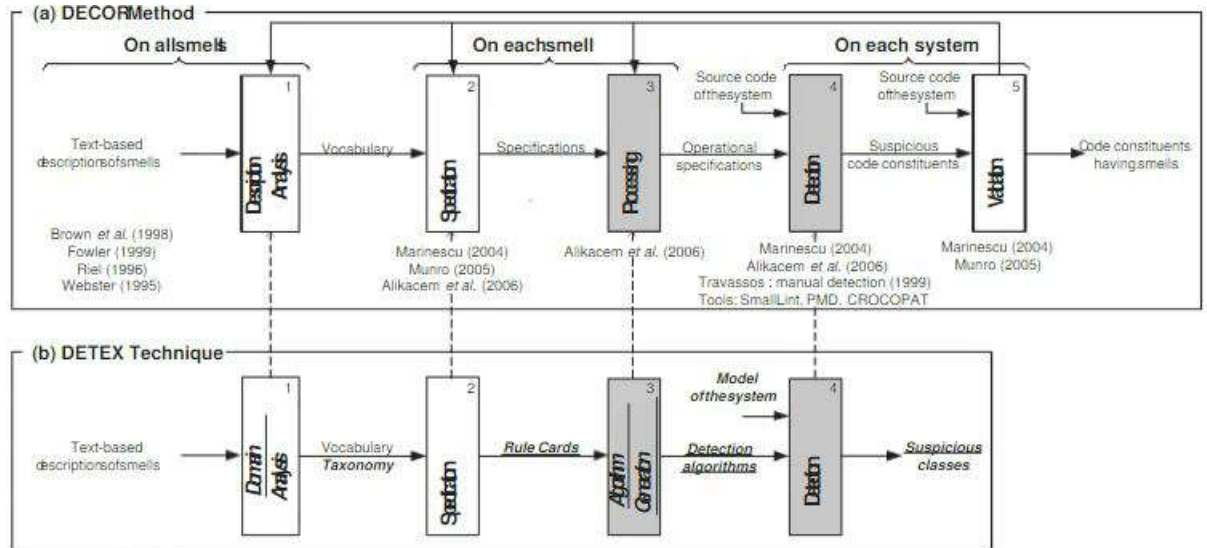


Figura 3.2: Le tecniche di rilevamento di DECOR e DETEX comparate

3.3 Il funzionamento di TACO

TACO viene utilizzato per l'identificazione automatica di code smell, attraverso i principi di Information Retrieval (IR), studiati da Palomba et al.. Esso è stato implementato da cASpER al fine di supportare l'identificazione di quattro tipi di code smell: *Promiscuous Package*, *Misplaced Class*, *Blob* e *Feature Envy*.

3.3.1 Individuazione dei code smell tramite l'analisi testuale

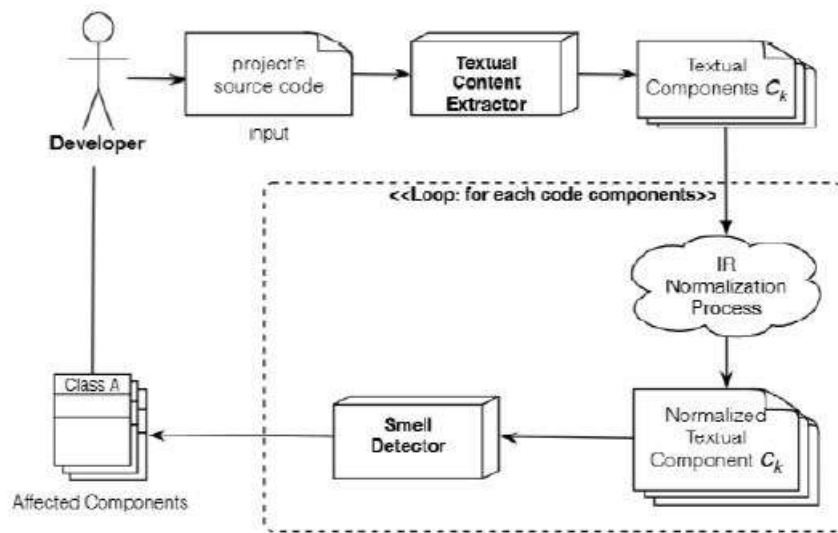


Figura 3.3: Il processo di individuazione di uno smell da parte di TACO

Processo di identificazione

Il processo di identificazione di code smell si divide in tre fasi[8]: Textual Content Extractor, IR Normalization, Smell Detector.

Textual Content Extractor. L'obiettivo della prima fase di analisi consiste nell'estrarre da ogni file sorgente del software che si sta sviluppando il suo contenuto testuale. Gli elementi scelti per tale estrazione corrispondono in generale ai commenti ed agli identificatori, in quanto questi rappresentano gli elementi realmente necessari per TACO.

IR Normalization Estratti commenti ed identificatori, questi sono sottoposti ad un processo detto di "normalizzazione", che li trasforma applicando in sequenza i passaggi così descritti:

- tramite uno splitting sul *camel case*, che taglia le parole in corrispondenza di trattini bassi o maiuscole o numeri, gli identificatori composti vengono separati;
- ogni lettera estratta è resa minuscola;
- ognuna delle più usate *stop word*, ogni parola chiave (di programmazione) ed ogni carattere speciale viene rimosso;
- tramite il "*Porter's Stemmer*"[9] le radici originali delle parole estratte vengono derivate a loro volta.

Una volta eseguiti questi passaggi si ottiene una lista di parole normalizzate; ora non resta che pesarle. A questo scopo ci si affida ad uno schema detto *term frequency inverse document frequency (tf-idf)*[8] il cui compito è ridurre la rilevanza delle parole troppo generiche contenute nella maggior parte delle componenti di partenza.

Smell Detector. Dopo la normalizzazione, lo *Smell Detector* applicherà sulle parole ottenute euristiche differenti per i vari smell ottenibili. Il detector tramite il Latent Semantic Indexing (LSI), estensione del Vector Space Model (VSM)[8], trasforma le componenti del codice in vettori di termini, poi il LSI[10] usa il Singular Value Decomposition (SVD)[11] per raggruppare le componenti in base alle relazioni tra parole ed altre componenti. Al fine poi di limitare l'effetto del rumore testuale, i vettori iniziali (ottenuti con il LSI) sono proiettati in un K-spazio ridotto le cui dimensioni sono determinate dalle euristiche di Kuhn[12]. Tale dimensione k è $k = (mxn)^{0.2}$, dove m rappresenta le dimensioni del vocabolario e n il numero di documenti. Successivamente si applica il coseno dell'angolo dei due vettori per ottenere la somiglianza tra i due documenti.

Poiché vi sono più possibili smell che potrebbero presentarsi, ogni valore è calcolato in modo diverso a seconda dello smell considerato, aumentando di fatto la probabilità che una precisa componente sia effettivamente affetta da un particolare smell. Le probabilità vengono convertite in valori booleani *True*, *False* che indichino se lo smell sia presente o meno.

3.3.2 I Code Smell e la loro identificazione

Blob. Si comincia l'analisi dei Code Smell che è possibile rilevare con cASpER parlando del Blob. Un sistema che comporta un tale problema presenta sempre una classe centrale che monopolizza le principali se non tutte le operazioni del software, mentre attorno ad essa vi sono altre classi create con il solo scopo di immagazzinare dati. Parliamo di una classe che detiene un potere ed una responsabilità troppo grandi, generalmente risultato di una distribuzione dei requisiti non ottimale. Si è constatato che tale problema è spesso frutto dello

sviluppo iterativo dei progetti software durante il quale un'incorretta ripartizione dei requisiti fa sì che una classe possa avere responsabilità troppo più grandi rispetto alle altre. Poiché spesso un Blob è caratterizzato da codice non necessario, che rende difficile comprendere con precisione quali siano le funzionalità che sono più utili alla classe, il refactoring che andrebbe eseguito è detto *Extract Class Refactoring*, il cui scopo è dividere il Blob in due o più classi con responsabilità più coese.

DECOR, nell'identificare lo smell, si concentra su euristiche che sfruttano metriche di qualità del software, come ad esempio la coesione. Infatti, definisce una così detta "rule card" (Figura 3.1) che descrive le peculiarità di una classe Blob e ciò che ne consegue per l'identificazione. DECOR etichetta come Blob una classe con un LCOM5 (Lack of Cohesion Of Methods)[4] superiore a 20, più di 20 tra attributi e metodi, un'associazione di tipo uno a molti con altre classi ed un nome che ha per prefisso: Process, Control, Command, Manage, Drive, System.

Si veda ora invece come TACO rileva i Blob. L'ipotesi che TACO sfrutta per l'identificazione dei Blob si concentra sulla loro caratteristica di possedere un'alta dispersione semantica. Sia C la classe da analizzare, $M = M_1, \dots, M_n$ l'insieme dei metodi in C , la coesione testuale della classe è calcolata come segue[14]:

$$ClassCohesion(C) = mean\ sim(M_i, M_j)$$

con i diverso da j , n è il numero di metodi in C e $sim(M_1, M_2)$ denota la somiglianza testuali tra i metodi M_1 ed M_2 in C . Dunque la probabilità che C sia un Blob è:

$$P_B(C) = 1 - ClassCohesion(C)$$

Il valore $P_B(C)$ sarà nell'intervallo $[0,1]$.

Promiscuous Package. Il *Promiscuous Package* è il corrispondente ma a livello di package del *Blob*. I package in generale raggruppano insieme le classi che sono logicamente e strutturalmente correlate e tali raggruppamenti, se non eseguiti in maniera adeguata, possono risultare in moduli difficilmente comprensibili e dunque difficilmente manutenibili. Quindi possiamo dire che un package con bassa coesione presenterà con alta probabilità uno smell di Promiscuous Package.

Il refactoring corrispondente è chiamato *Extract Package Refactoring*.

TACO identifica tale smell assumendo che package affetti contengano classi semanticamente molto distanti fra loro. Trasformando questo in formule: siano

P il package in esame e $C = C_1, C_2, \dots, C_n$ l'insieme delle sue classi. La coesione testuale del package P è rintracciata tramite la formula (Poshyvanyk[13]):

$$PackageCohesion(P) = mean\ sim(C_i, C_j)$$

con i diverso da j, n rappresenta il numero delle classi in P e $sim(C_i, C_j)$ è la somiglianza tra le classi C_i, C_j . Dunque non resta che definire la probabilità che P sia affetto dallo smell Promiscuous Package, tale probabilità è così calcolata:

$$P_{PP}(P) = 1 - PackageCohesion(P)$$

Tale probabilità avrà un valore che ricadrà all'interno dell'intervallo $[0,1]$.

Feature Envy. Un altro fondamentale smell preso in esame da cASpER è il Feature Envy che si ha quando "un metodo è più interessato ad una classe diversa rispetto a quella in cui si trova" (definizione di Fowler) (aggiungi bibliografia). Questa caratteristica si traduce nella rilevazione di un livello di accoppiamento tra il metodo e la sua classe di appartenenza minore rispetto a quello che ha nei confronti di una classe esterna.

Il *Move Method Refactoring*[5] si occupa di spostare il metodo nella classe considerata più adatta, risolvendo in questo modo il problema. Come fa TACO ad identificare lo smell?

Taco ipotizza per prima cosa che il metodo preso in esame abbia una maggiore somiglianza e semantica e concettuale con una classe diversa dalla propria e per questo detta *envied* (in italiano "invidiata"). Formalizzando tale concetto siano M il metodo, C_0 la classe di appartenenza e $C_{related} = C_1, \dots, C_n$ l'insieme delle classi che condividono almeno un concetto con M. La classe invidiata $C_{closest}$ è calcolata tramite l'applicazione della seguente formula:

$$C_{closest} = arg_{C_i \in C_{related}} max\ sim(M, C_i)$$

dove $sim(C_i, C_j)$ indica la somiglianza testuale tra il metodo X e la classe Y. Se $C_{closest}$ è diversa da C_0 allora M va spostato in un'altra classe. Le probabilità che M sia affetto da *Feature Envy* è calcolata come:

$$P_{FE} = sim(M, C_{closest}) - sim(M, C_0)$$

P_{FE} è chiaramente 0 quando $C_0 = C_{closest}$ e cioè quando il metodo è posizionato correttamente.

Misplaced Class. Quando si parla di *Misplaced Class* si intende riferirsi a

una classe contenuta in un package con classi che però sono, concettualmente parlando, molto lontane da essa.

L'operazione di refactoring necessaria per la correzione di un tale errore di progettazione è detta *Move Class Refactoring*, il cui compito è quello di spostare la classe dal suo attuale package ad uno che più gli è idoneo. Anche qui è TACO a prendere in carico questo problema, identificandolo confrontando la semantica della classe in esame con la semantica del package a cui appartiene.

Si formalizza come per gli altri questo concetto, definendo C quale la classe analizzata, P_0 è invece il package di appartenenza di C , sia poi $P_{related} = P_1, \dots, P_n$ l'insieme dei package che condividono almeno un termine con C . La prima operazione da effettuare è il recupero del package che più degli altri si addice a C (ossia che più gli assomiglia dal punto di vista testuale), tramite la seguente formula:

$$P_{closest} = \arg_{P_i \in P_{related}} \text{sim}(C, P_i)$$

Se $P_{closest}$ è diverso da P_0 allora C dovrebbe essere spostata nel package $P_{closest}$. Più precisamente, le probabilità che C sia affetta da *Misplaced Class* è calcolata come:

$$P_{MC} = \text{sim}(C, P_{closest}) - \text{sim}(C, P_0)$$

Si ha che $P_{MC} = 0$ quando $P_{closest} = P_0$, altrimenti sarà pari alla differenza tra le somiglianze testuali tra C ed i due package $P_{closest}$ e P_0 .

3.4 Funzionamento di cASpER per IntelliJ IDEA

I due approcci esaminati precedentemente per il rilevamento di code smell sono stati implementati parallelamente per lo sviluppo del plugin cASpER distribuito per il noto IntelliJ IDEA.

Di fondamentale importanza ai fini della comprensione del lavoro svolto nell'ambito della presente tesi, che verrà descritto nel successivo capitolo, risulta un'analisi più approfondita riguardo alle funzionalità principali proposte da cASpER; di quest'ultimo sarà poi analizzata anche l'architettura scelta. Per concludere verrà mostrato un esempio di utilizzo di cASpER.

3.4.1 Le funzionalità di cASpER

cASpER è composto da tre casi d'uso principali: configurazione dei thresholds, analisi globale del progetto ed ispezione dei singoli code smell rilevati. Quest'ultimo si specifica in quattro ulteriori casi d'uso, per un totale di sei funzionalità da analizzare. Ecco dunque i casi d'uso (Figura 3.4):

- **Configure Thresholds.** Tramite questo caso d'uso, lo sviluppatore è in grado di configurare manualmente i parametri di thresholds, in modo che siano più adatti possibile al proprio progetto;
- **Check Project.** Lo sviluppatore, tramite questa funzione, lancia l'analisi globale di code smell che analizzerà l'intero progetto. Al termine dell'analisi, cASpER mostrerà una tabella con l'elenco dei difetti rilevati ed una serie di ulteriori parametri, tra i quali anche il tipo di code smell identificato.
- **Inspect Promiscuous Package.** Dopo l'esecuzione della funzionalità Check Project, lo sviluppatore, selezionando un code smell di tipo *Promiscuous Package*, può avviarne l'ispezione. Il sistema procede mostrando i risultati dell'ispezione e le *Radar Map* formate dai cinque termini più frequenti che rappresentano i topic implementati. A questo punto l'utente ha la possibilità di correggere automaticamente lo smell proseguendo nell'interfaccia; il plugin quindi gli mostrerà le modifiche che verranno eseguite nel caso si lanciasse l'operazione di refactoring. Nel caso del Promiscuous Package, il plugin mostrerà la modalità di divisione dei package in due o più package minori, con maggiore coesione, indicando i relativi topic. Lo sviluppatore può quindi avviare l'operazione di correzione; inoltre potrà, prima del refactoring, modificare le metriche strutturali per effettuare un ricalcolo dell'algoritmo di divisione.
- **Inspect Misplaced Class.** Dopo l'esecuzione della funzionalità Check Project, lo sviluppatore, selezionando un code smell di tipo *Misplaced Class*, può avviarne l'ispezione. A questo punto viene mostrata la *Radar Map* della classe analizzata, le *Radar Map* dei relativi package di appartenenza ed *Envied*. Procedendo verso il refactoring, si mostrano all'utente le modifiche che si prevede di effettuare proponendo anche le *Radar Map* dei corrispondenti package dopo l'applicazione del *Move Class Refactoring*. Dunque non resta che lanciare la correzione.
- **Inspect Blob.** Dopo l'esecuzione della funzionalità Check Project, lo sviluppatore, selezionando un code smell di tipo *Blob*, può avviarne l'ispezione. Avviata l'ispezione, saranno visualizzate: una tabella contenente le

metriche della classe affetta, il suo contenuto testuale e la *Radar Map* dei topic da essa implementati. A questo punto, seguendo l'interfaccia, verrà proposta l'operazione di refactoring detta *Extract Class Refactoring* in cui sarà presente anche il numero, variabile in base al tipo di analisi effettuata ed alle metriche scelte, di suddivisioni consigliato. Eventualmente i nomi delle classi che saranno ottenute potranno essere modificate dall'utente, così come le posizioni dei metodi al loro interno. Infatti il refactoring di cui si parla formerà classi all'interno delle quali spostare i metodi che le sono assegnati. Questa operazione si affida all'utilizzo di una componente detta *Delegation*, il cui compito consiste nel copiare il metodo nella nuova classe e modificare in maniera opportuna la vecchia versione del metodo all'interno del Blob, il quale conterrà la chiamata al nuovo metodo. Terminato il refactoring, saranno presenti nuove classi ma il comportamento del sistema rimarrà il medesimo.

- **Inspect Feature Envy.** Dopo l'esecuzione della funzionalità Check Project, lo sviluppatore, selezionando un code smell di tipo *Feature Envy*, può avviarne l'ispezione. Avviata l'ispezione, il plugin mostrerà due diverse *Radar Map*: una relativa alla classe affetta, affiancata da una tabella contenenti i metodi affetti, l'altra riguardante uno dei metodi della classe, abbinata al codice del metodo. Procedendo al refactoring, allo sviluppatore sarà mostrata una schermata contenente i metodi affetti dallo smell, le *Radar Map* relative allo stato attuale delle classi d'origine e proposte quelle delle stesse classi dopo il refactoring. A questo punto non resta che avviare la correzione.

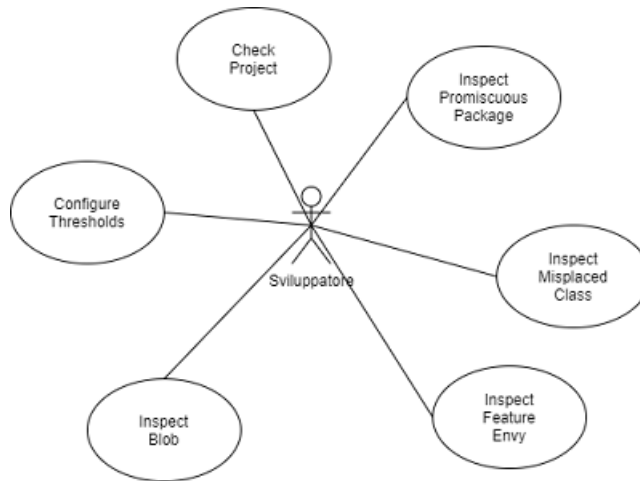


Figura 3.4: Diagramma dei casi d'uso di cASpER

3.4.2 Architettura di cASpER

cASpER presenta un'architettura di tipo three-tier, infatti sono presenti i classici tre livelli: *Interface Layer*, *Application Layer* e *Storage Layer*. Di seguito li si analizzeranno più nel dettaglio.

Interface Layer. Questo livello comprende in generale tutte le componenti che si occupano della visualizzazione delle finestre che compongono l'interfaccia grafica del plugin. In particolare si parla delle classi contenute nel package *casper.gui*. Queste componenti si occupano di mostrare le pagine di ispezione, refactoring ed analisi del progetto in esame. In particolare fanno uso delle classi contenute nel package *casper.gui.radarMap*.

Application Layer. Questo livello include tutte le componenti relative agli oggetti di gestione ed alle entità del sistema. In particolare si concentra sulle classi relative all'identificazione dei code smell ed al refactoring. Di primaria importanza sono i package *casper.analysis.code_smell_detenction*, che contiene le classi ed i package relativi al rilevamento degli smell, ed il package *casper.refactor* che invece contiene classi e package relativi alle operazioni di refactoring.

Storage Layer. Poiché alcuni parametri una volta impostati devono essere memorizzati, è stato necessario implementare un ulteriore livello di memorizzazione dei valori scelti; ciò ne consente il recupero e l'interrogazione in maniera del tutto trasparente.

3.4.3 Esempio di utilizzo di cASpER

Il primo passo da eseguire per avviare il plugin è quello di lanciare la funzionalità di **Check Project**. Al termine dell'analisi, verrà mostrata una finestra contenente, oltre alle altre informazioni, una tabella composta dalla lista dei code smell identificati (Figura 3.5).

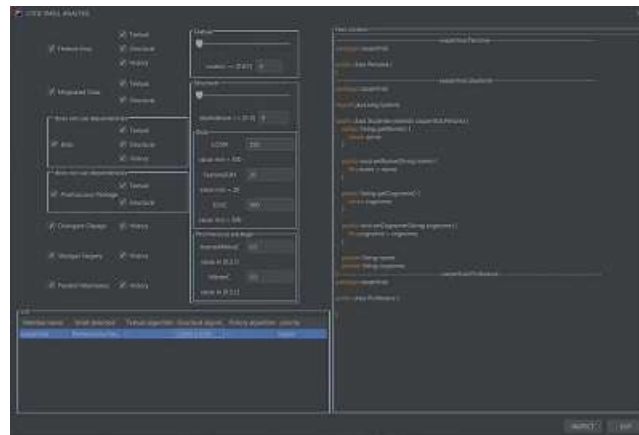


Figura 3.5: Finestra di Check Project

Selezionato il code smell di tipo *Promiscuous Package* e cliccato su *Inspect*, il plugin porterà su una seconda schermata nella quale verranno mostrati i risultati dell'ispezione (Figura 3.6).

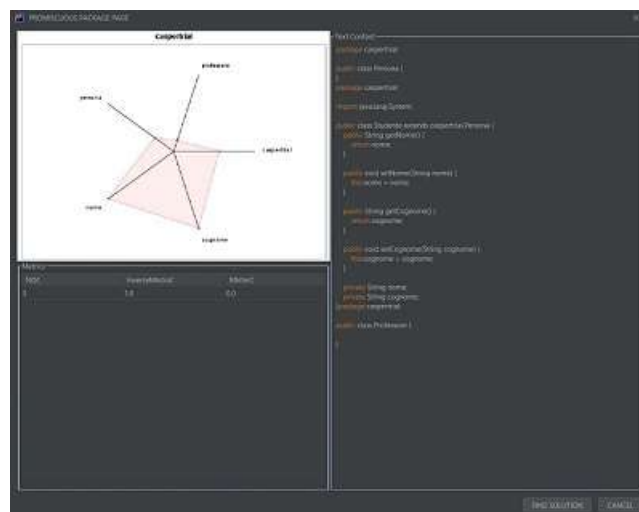


Figura 3.6: Finestra di ispezione

Cliccando poi su *Find Solution* cASpER presenterà la soluzione consigliata, con i rispettivi risultati che ne conseguiranno (Figura 3.7). A questo punto non ci resta che cliccare su *Refactoring* per risolvere il problema.

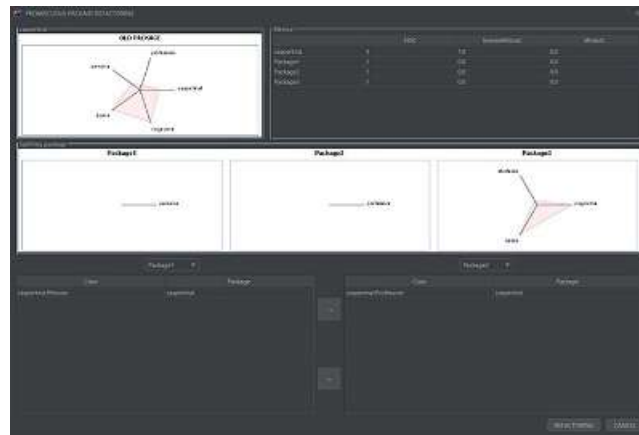


Figura 3.7: Finestra di refactoring

Capitolo 4

cASpER per la Continuous Integration

Dopo aver discusso nei precedenti capitoli di Continuous Integration e di rilevamento di *code smell*, risulta subito evidente come il connubio tra questi due fondamentali concetti di Ingegneria del Software possa apportare interessanti benefici al ciclo di vita di un sistema software.

Proprio da questa idea si muove il lavoro di tesi che ci si accinge a descrivere, il cui scopo principale è stato più nello specifico quello di integrare il plugin cASpER, per il rilevamento automatico di *smell*, nel contesto della Continuous Integration; quest'ultimo usufruito tramite la piattaforma Travis CI. Più precisamente, grazie al lavoro svolto è stato reso possibile eseguire il plugin cASpER su ogni porzione di codice che venga progressivamente caricata nella repository centrale del progetto che si sta realizzando, al fine di avvertire gli sviluppatori riguardo alla presenza di *smell* all'interno del lavoro che hanno svolto.

Al fine di realizzare questo obiettivo si è reso necessario svincolare il plugin dall'ambiente per il quale è stato inizialmente realizzato, ovvero IntelliJ IDEA. Fondamentale è stata dunque la reimplementazione del componente parser di cASpER al fine di rompere le dipendenze con la piattaforma IntelliJ IDEA. Inoltre, per ottenere una maggiore praticità di integrazione con Travis CI, è stata disposta una nuova componente che permetta di eseguire cASpER da linea di comando.

Nei capitoli successivi verrà descritto nello specifico il processo di migrazione di cASpER da IntelliJ IDEA (quindi con l'adozione da parte del plugin di un nuovo parser) Ci si soffermerà, inoltre, su cosa è stato implementato *ex novo* e cosa invece è stato riutilizzato affinché il sistema continuasse ad offrire le medesime funzionalità di rilevamento di *smell* descritte nella sezione 3.4. Verranno inoltre presentate tutte le difficoltà incontrate durante tale processo e le

rispettive soluzioni che sono state implementate, nonché le alternative prese in considerazione. Infine sarà mostrato un piccolo esempio di utilizzo del sistema finale.

Tuttavia, ai fini di questo lavoro, non è stato ritenuto necessario implementare *ex novo* le operazioni di refactoring già presenti nella versione originale del plugin cASpER. A motivare questa scelta vi è la volontà di salvaguardare la sicurezza del progetto software al quale si sta lavorando; infatti effettuare modifiche al codice sorgente da fonti esterne, qual'è Travis CI, potrebbe risultare estremamente dannoso per lo sviluppatore o addirittura per l'intero team di sviluppo. Implementare le operazioni di refactoring è dunque un compito che va eseguito con estrema minuziosità e che quindi si prevede possa essere preso in considerazione per sviluppi futuri.

4.1 Componenti Riutilizzate e Rimosse

Di seguito è mostrato tramite la Figura 4.1 il diagramma UML che mostra la struttura della versione di cASpER su cui è basato il lavoro di tesi. Tale diagramma contiene dunque sia le componenti che sono state riutilizzate sia quelle che sono state rimosse, ossia quelle che non si sono rese necessarie per lo sviluppo della nuova versione di cASpER.

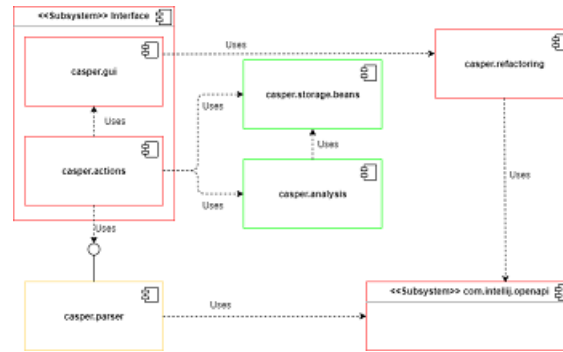


Figura 4.1: Struttura della precedente versione del plugin cASpER

In rosso le componenti eliminate

In giallo le componenti riutilizzate con modifiche

In verde le componenti riutilizzate senza modifiche

Segue una descrizione delle componenti rappresentate nella Figura 4.1:

- **casper.gui** All'interno di questa componente sono racchiuse tutte le classi che permettono la realizzazione delle varie finestre dell'interfaccia grafica del plugin. Non è stato necessario mantenere questa componente nella nuova versione di cASpER in quanto la piattaforma che si intende utilizzare per lanciare il plugin, ossia Travis CI, prevede il solo utilizzo di un'interfaccia testuale. Ecco perché *casper.gui* è stata sostituita da *casper.cli*.
- **casper.actions** Per poter realizzare le proprie funzionalità, il codice di un plugin deve essere invocato da un qualche metodo all'interno del sistema che esso estende. Tali metodi, nella vecchia versione di cASpER, sono presenti all'interno di *casper.actions*. Anche questa componente però è stata rimossa nella nuova versione del plugin poiché strettamente dipendente dal sottosistema *casper.gui* sopra citato. Nella nuova versione del plugin, il compito di invocare il codice di cASpER è stato affidato esclusivamente al package *casper.cli* che sarà discusso nella sezione successiva, rendendo non ulteriormente necessario l'utilizzo di *casper.actions*.
- **casper.parser** Questa componente corrisponde alla vecchia versione del parser di cASpER ed è stato modificato quasi del tutto. Il suo funzionamento, le differenze con la nuova versione del parser e le motivazioni che hanno portato a tali modifiche saranno mostrate nella sezione successiva.
- **casper.refactoring** Questa componente contiene le classi che implementano le operazioni di refactoring del codice. Essa è stata rimossa nella nuova versione di cASpER in quanto, come si è già detto al termine della precedente sezione, prevedendo di lanciare il plugin da una piattaforma esterna rispetto all'ambiente nel quale risiede il codice del progetto a cui si lavora, la sua applicazione potrebbe risultare dannosa.
- **casper.storage.beans e casper.analysis** Queste due componenti sono state riutilizzate nella nuova versione di cASpER e ci si appresta a descriverle nel dettaglio.

Le principali componenti del progetto iniziale di cASpER che sono state lasciate inalterate sono due package di fondamentale importanza: *casper.storage.beans* e *casper.analysis* (compresi i package a quest'ultimo direttamente connessi). Tale riutilizzo è stato possibile in quanto entrambi i package sono stati realizzati in maniera completamente indipendente rispetto ad IntelliJ IDEA. Tra i due package sopra menzionati, il secondo rappresenta senza dubbio il cuore pulsante dell'analisi del plugin: esso, infatti, contiene tutte le classi utili ai fini dell'individuazione automatica degli smell.

Ci si appresta dunque a descrivere più nel dettaglio i due package riutilizzati.

casper.storage.beans cASpER analizza code smell a livello di package, classi, metodi e variabili d'istanza, ecco perché per effettuare tali analisi ha bisogno di particolari classi dette beans. Questi ultimi rappresentano infatti i quattro tipi di elementi fondamentali del linguaggio di programmazione Java sopra menzionati. In particolare, alcune delle informazioni che i beans mantengono sono relative a: nome, contenuto testuale, linee di codice, invocazione di metodi e così via. Proprio sulla base delle informazioni raccolte nei beans, comincia l'azione delle componenti implementate all'interno del package *casper.analysis*, tramite le quali saranno svolti i calcoli di analisi strutturale e testuale del codice implementato, secondo le tecniche descritte da DECOR e TACO (vedi sezione 3.2 e sezione 3.3). Dunque le classi del package *casper.storage.beans* rappresentano la base per il lavoro dell'intero plugin, ecco perché è stato di fondamentale importanza poter riutilizzare tale componente.

casper.analysis Questo package rappresenta senza alcun dubbio il cuore pulsante del plugin cASpER, anche se in realtà esso è composto da tre package ausiliari che sono: *casper.analysis.code_smell*, *casper.analysis.code_smell_detention*. Tra le più importanti classi presenti all'interno dei package citati vi sono: *BlobCodeSmell*, *FeatureEnvyCodeSmell*, *MisplacedClassCodeSmell*, *PromiscuousPackageCodeSmell*. Queste classi hanno il compito di rappresentare i diversi tipi di code smell le cui analisi e correzione sono supportate da cASpER. Altre componenti fondamentali che ritroviamo all'interno di *casper.analysis* sono *StructuralBlobStrategy*, *TextualBlobStrategy*, *StructuralFeatureEnvyStrategy*, *TextualFeatureEnvyStrategy* ecc..., il cui fine è invece quello di implementare le strategie strutturali e testuali di analisi automatica, definite da DECOR e TACO (sezione 3.2 e sezione 3.3), dei corrispondenti code smell.

Appare molto utile entrare più nel dettaglio riguardo alla suddivisione del package *casper.analysis*, permettendo in questo modo una maggiore comprensione dei suoi compiti. I compiti dei package contenuti in esso sono:

- ***casper.analysis.code_smell*** Questo package contiene le classi, precedentemente citate, che individuano i code smell supportati da cASpER. In particolare, risulta di maggiore importanza concentrarsi sulla componente *CodeSmell*, la quale rappresenta la classe di partenza[6], in quanto astratta, da cui le altre derivano e che quindi definisce la struttura di base degli altri smell. Più nel dettaglio, da *CodeSmell* derivano direttamente le classi *ClassLevelCodeSmell*, *PackageLevelCodeSmell*, *MethodLevelCodeSmell*, anch'esse astratte, che formano la struttura iniziale delle classi che rappresentano gli smell veri e propri.

- ***casper.analysis.code_smell_detection*** Questo è sicuramente il package più complesso ed anche quello più importante, esso contiene tutte le classi dedicate all'analisi automatica degli smell. In particolare, ha il compito di implementare le metriche definite dalle tecniche espresse da DECOR e TACO (sezione 3.2 e sezione 3.3). Infatti esso è suddiviso in tanti package quanti sono gli smell supportati, all'interno dei quali sono contenute le classi che stabiliscono le strategie strutturali e testuali di analisi.

4.2 Nuove Componenti Implementate

Con l'aiuto di un UML component diagram, evidenziando le componenti ed i sottosistemi implementati, è stato possibile mostrare in maniera chiara la nuova architettura di cASpER, che viene descritta nella Figura 4.2. Dunque, consultando la Figura 4.1 che mostra la struttura della precedente versione di cASpER, sono subito evidenti alcune differenze tra le due versioni del plugin. Infatti, nella nuova versione di cASpER sono presenti delle componenti totalmente nuove, quali: *casper.cli*, *casper.adapters*, *casper.parser*, *casper.beans*. Per comprendere però al meglio le funzionalità delle nuove componenti introdotte, segue una descrizione più dettagliata.

Nel sottosistema identificato come "Interface" è stata presentata la componente "ComandLineCasper" che implementa la nuova interfaccia di cui si serve cASpER. Infatti, affinché il plugin sia fruibile tramite Travis CI, si è reso necessario un passaggio dall'interfaccia grafica all'interfaccia testuale. Nella Figura 4.2 è possibile vedere anche il sottosistema relativo al parser del plugin. Anche in questo caso si tratta di una componente totalmente nuova, ciò è stato dovuto dal fatto che il parser usato dalla precedente versione di cASpER presentasse dipendenze con IntelliJ IDEA, rendendo dunque impossibile il suo utilizzo al di fuori di tale ambiente. Il sottosistema *casper.adapters* contiene le classi che hanno permesso, tramite l'applicazione del design pattern *Adapter*, la conversione dalle componenti utilizzate dalla precedente versione del parser alle componenti richieste dalla sua nuova versione. Nel diagramma è possibile notare anche il sottosistema *casper.beans* che contiene le nuove versioni dei bean utilizzate dal parser. Infine nel diagramma è stato inserito anche il sottosistema *org.eclipse.jdt* che individua tutte le API messe a disposizione dalla piattaforma per il controllo e l'analisi del codice da parte del nuovo parser. A seguire saranno analizzate più nel dettaglio tutte le componenti implementate di cui abbiamo appena discusso.

- ***casper.cli*** Affinché il plugin potesse essere lanciato tramite Travis CI, il cui funzionamento è stato descritto nella sezione 2.3, è stato necessario realizzare una classe principale che implementasse la funzione statica *main*. Tale classe è stata denominata *CommandLineCasper* ed è stata inserita all'interno del package *casper.cli*. Le responsabilità di questa classe

partono con la ricerca dei package contenuti all'interno del progetto sotto esame, il cui percorso viene specificato come unico argomento sulla linea di comando. Una volta ricavati tutti i package contenuti nel progetto, si procede con la loro analisi, tramite il nuovo parser, al termine della quale sarà disponibile la lista di package, class e method bean con, eventualmente, i relativi code smell rilevati. La scelta del package *casper.cli* ovviamente non è stata casuale in quanto la componente `CommandLineCasper` è stata realizzata anche in modo da implementare un'interfaccia testuale da riga di comando. Infatti poiché Travis CI non permette l'utilizzo di interfacce grafiche ma fornisce solo l'uso di un'interfaccia testuale, al fine di poter notificare lo sviluppatore riguardo la presenza di bad code smell all'interno del codice del progetto che sta realizzando, è stato fondamentale poter convertire la tabella che nella presentazione grafica della precedente versione del plugin cASpER elencava i code smell rilevati, in un elenco visualizzabile semplicemente per via testuale all'interno di un'interfaccia testuale.

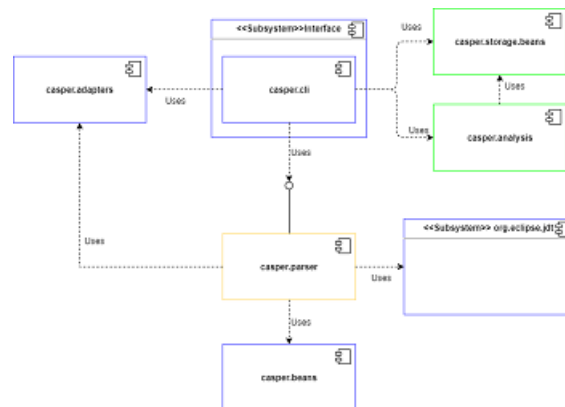


Figura 4.2: Struttura aggiornata del plugin cASpER

In giallo le componenti riutilizzate con modifiche.

In verde le componenti riutilizzate senza modifiche.

In blu le componenti nuove.

- **casper.parser** Questo package era in realtà già presente nella precedente versione del plugin cASpER per IntelliJ IDEA ma ne è stata necessaria una riscrittura, in quanto era strettamente accoppiato alle API della piattaforma di IntelliJ IDEA.

Il compito che deve eseguire il parser di cASpER è di fondamentale importanza; esso deve infatti creare, a partire dal relativo codice sorgente, i package, class e method bean che permettono il rilevamento dei code smell. Infatti, è proprio a partire da tali bean che verranno effettuate le analisi da

parte del package *casper.analysis* di cui abbiamo già ampiamente discusso nella sezione precedente.

Come già detto, la vecchia versione del parser era fortemente accoppiata ad IntelliJ IDEA. Più nel dettaglio, gli elementi del codice sorgente (classi, metodi, ecc.) venivano rappresentati tramite una struttura chiamata PSI (Program Structure Interface), un layer il cui compito era quello di effettuare il parsing dei file per poi creare la struttura semantica e sintattica del codice. Senza alcun dubbio, risulta di estrema importanza la componente **PSIFile**, che corrisponde alla radice di una struttura che rappresenta un file tramite una gerarchia di elementi, detti **PSIElement**. Ad ogni elemento PSI è associata una gerarchia che contiene strutture PSI che rappresentano i vari elementi del codice sorgente. Quindi i metodi, le classi, le variabili d'istanza ed i package sono visti come specializzazione di elementi PSI, poiché tali elementi PSI e le operazioni a livello dei singoli elementi sono usati per analizzare la struttura del codice sorgente così come è interpretata da IntelliJ stesso.

Da qui si è resa necessaria la realizzazione di una nuova versione del parser la quale invece utilizza le funzionalità fornite dall'API *org.eclipse.jdt*, il quale usufruisce delle procedure di JDT che consentono la visita dell'Abstract Syntax Tree (AST) del progetto in analisi al fine di ricavarne i bean. Dunque le classi del nuovo parser sono state realizzate in modo da eseguire le visite all'interno dell'AST.

Poiché il parser è suddiviso in tante componenti quanti sono gli elementi di cui vogliamo effettuare l'analisi (ossia classi, package, metodi e variabili d'istanza), doveva essere trovata un'architettura che potesse organizzare al meglio tale decomposizione. Ecco perché è stata scelta come architettura del parser una classica implementazione della soluzione proposta dal design pattern *Facade*. Si tratta infatti di un'architettura che presenta un'interfaccia centrale che delega le chiamate del parsing alla classe corrispondente alla componente che si vuole analizzare in un dato momento.

Uno dei problemi riscontrati durante questo lavoro corrisponde al fatto che mentre il package *casper.analysis* utilizza le vecchie versioni dei bean (di cui si è discusso nella sezione precedente, contenuti nel package *casper.storage.bean*), i risultati delle nuove operazioni di parsing corrispondono alla nuova versione dei bean (contenuti in *casper.beans*). Dunque, la nuova versione del parser prevede di effettuare, prima di lanciare le procedure di analisi, una conversione dei bean dalla nuova alla vecchia versione.

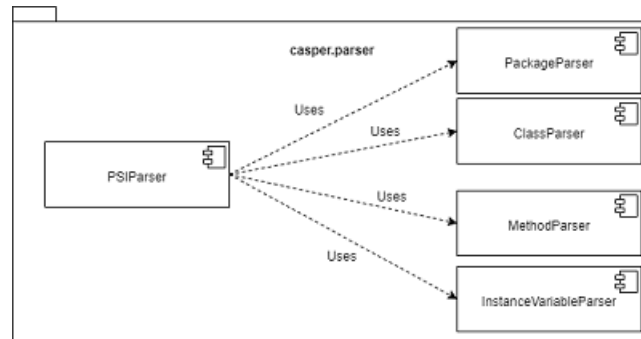


Figura 4.3: Struttura aggiornata del parser

- **casper.adapters** Il passaggio dalla vecchia alla nuova versione del parser ha comportato una serie di problemi di conversione. Infatti le interfacce e le classi che venivano utilizzate nella versione precedente sono completamente diverse rispetto a quelle utilizzate nella versione attuale. Ecco perché è stato necessario trovare una soluzione al problema di compatibilità. A tal fine si è deciso di implementare la soluzione descritta dal design pattern Adapter, implementando in questo modo delle interfacce che implementassero le interfacce richieste dai nuovi metodi di parsing.

Centrale in questo caso è stato il problema riscontrato con il parsing dei package, dal quale dipendono i parsing di classi, metodi e variabili d'istanza. Infatti, la nuova versione del PackageParser richiede il passaggio di un parametro di tipo `IPackageFragment`, un'interfaccia di *org.eclipse.jdt* che rappresenta un'intero package. Il problema consisteva nel fatto che all'avvio del plugin, `CommandLineCasper` effettua, come già detto, un'analisi dell'intero progetto che si sta sviluppando col fine di ottenere una lista di package, i quali però in partenza sono rappresentati tramite una struttura che non è compatibile con `IPackageFragment`, chiamata `GeneralPackage`. La strategia che si è ritenuta più utile e che si è implementata ha fatto sì che ogni package ricavato durante la ricerca e dunque incapsulato all'interno della componente `GeneralPackage`, venisse convertito in un elemento del tipo `PackageAdapter`. Quest'ultimo non è che una classe che implementa l'interfaccia `IPackageFragment`, in questo modo la componente `PackageParser` può effettuare il parsing del package senza alcun problema. La Figura 4.4 descrive tramite un diagramma UML la struttura della soluzione appena descritta.



Figura 4.4: Soluzione che utilizza il design pattern Adapter

La presenza di diverse classi adapter all'interno di *casper.adapter* è dovuta al fatto che le classi scelte come target della conversione contengono a loro volta delle dipendenze con interfacce non supportate dalla precedente versione del plugin; è stato quindi d'obbligo implementare adapter che implementassero queste dipendenze.

- **casper.bean** Si è considerato necessario includere all'interno del progetto di cASpER anche questo package, in quanto esso contiene tutti i bean che vengono istanziati dal nuovo parser.

Il problema legato a questo package riguarda il fatto che i bean che vi sono all'interno, a differenza di quelli che si trovano all'interno del package *casper.storage.beans*, non contengono informazioni riguardo i code smell che sono stati rilevati all'interno delle componenti che rappresentano.

Per questo motivo, prima di effettuare il rilevamento di eventuali smell, i bean ottenuti tramite la nuova versione del parsing, sono convertiti nei corrispondenti bean che si trovano all'interno del package *casper.storage.beans*.

4.3 Integrazione in Travis CI

Poiché l'obiettivo principale del lavoro di tesi è consistito nell'integrare cASpER nell'ambito della Continuous Integration, è stano necessario implementare, al fine di testarne il funzionamento, all'interno della repository di un progetto di prova, il file *travis.yml*.

Come già discusso nella sezione 2.3, il file *travis.yml* ha il compito, tra gli altri, di definire la pipeline di test che Travis CI dovrà eseguire. All'interno del documento è stata dunque inserita la chiamata per l'esecuzione del file "casper-1.1.2.jar" che rappresenta la nuova versione del plugin cASpER, ottenuta tramite l'applicazione delle soluzioni descritte nella precedente sezione.

Capitolo 5

Un esempio di utilizzo

Nel capitolo precedente sono state analizzate nel dettaglio tutte le operazioni che hanno permesso la realizzazione di una nuova versione del plugin cASpER che non fosse più dipendente in alcun modo da IntelliJ IDEA. A tale scopo è stato descritto ciò che è stato re-implementato, ciò che è stato riutilizzato e ciò a cui si è rinunciato. All'interno di questo capitolo verrà mostrato un esempio di utilizzo del plugin, al fine di rendere più chiare le modalità di esecuzioni relative al servizio messo a disposizione da Travis CI. Nella sezione a seguire sarà descritto infatti lo scenario relativo al rilevamento di *bad code smell* all'interno del progetto che si sta analizzando.

5.1 Rilevamento di code smell

La prima operazione che va eseguita per poter usufruire della pratica della Continuous Integration tramite Travis CI all'interno di un progetto a cui si sta lavorando consiste nel configurare il più volte menzionato documento *travis.yml*.

Si descrivono ora, con maggior precisione, gli elementi che compongono tale documento nel nostro esempio (Figura 5.1):

- **language.** Questo parametro indica il linguaggio di programmazione del progetto che si sta sviluppando;
- **jdk.** Quest'altro parametro indica la versione del jdk con il quale effettuare la build del progetto.
- **script.** Tale parametro è probabilmente il più importante, esso indica la sequenza di test che si ha intenzione di effettuare ogni volta che viene eseguito un push verso la repository centrale del progetto a cui si sta

lavorando. Nell'esempio in questione, per poter avviare il rilevamento di code smell, tramite la nuova versione del plugin cASpER, è stata inserita l'istruzione necessaria ad eseguire il file "casper-1.1.2.jar" con l'aggiunta di un parametro che indica il percorso relativo al progetto che si vuole analizzare.

```
language: java
jdk:
  - oraclejdk14
script:
  - java -jar cASpER-master/Code/build/libs/casper-1.1.2.jar HELLOWORLD
```

Figura 5.1: Esempio di file travis.yml

Una volta configurato tutto il necessario per l'utilizzo di Travis CI non resta che eseguire una semplice operazione di Push verso la repository del progetto da analizzare. A questo punto, recandosi nella pagina di gestione della Continuous Integration di Travis CI relativa al progetto in esame, sarà possibile controllare le operazioni che Travis CI eseguirà. Esso infatti, dopo aver effettuato la build del progetto, lancerà il file "casper-1.1.2.jar", come indicato dal parametro *script* all'interno del file *travis.yml*. Il risultato dell'esecuzione di cASpER tramite Travis CI è mostrato nella Figura 5.2.

Come si può vedere nella Figura 5.2, il plugin cASpER, una volta lanciato sul progetto corrispondente al percorso "HELLOWORLD", avendo individuato code smell, effettua due operazioni fondamentali:

- In primis, la nuova versione di cASpER è stata implementata in modo da mostrare un elenco dei code smell rilevati. Lo scopo di questo elenco è sostituire la tabella che, nella precedente versione del plugin, veniva presentata tramite interfaccia grafica in seguito all'esecuzione del rilevamento.
- Successivamente cASpER, per ogni code smell identificato, mostra allo sviluppatore le modifiche che verrebbero disposte se decidesse di effettuare i refactoring consigliati. Si ricorda però che attualmente tali operazioni di correzione sono eseguibili solo tramite l'utilizzo della precedente versione di cASpER disponibile per IntelliJ IDEA.

```
194 $ java -jar cASpER-master/Code/build/libs/casper-1.1.2.jar HELLOWORLD
195 Imposto i dati necessari
196 Ecco i code smell rilevati:
197 1 -> Member name: caspertrial | Smell detected: Promiscuous Package.
198
199 Package che si formeranno dopo il refactoring:
200 Package1
201 Package2
202 Package3
203 FINE
204 The command "java -jar cASpER-master/Code/build/libs/casper-1.1.2.jar HELLOWORLD" exited with 0.
```

Figura 5.2: Esempio di esecuzione di cASpER tramite Travis CI

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Conclusioni

Mantenere una buona qualità del software che si sta sviluppando e contemporaneamente rispettare le scadenze prefissate per le consegne sono obiettivi che, senza l'aiuto di strumenti esterni, risultano molto difficilmente raggiungibili. Inoltre la sincronizzazione dei lavori svolti dai vari sviluppatori coinvolti in un progetto non è così semplice, a causa delle problematiche di varia natura che possono manifestarsi durante la fase di integrazione. Al fine di assicurare la buona qualità del software e garantire il lavoro sincronico dei programmatori, è stata sviluppata una nuova versione del plugin cASpER, per il rilevamento di code smell all'interno di un progetto software, che possa essere eseguito tramite la piattaforma Travis CI e che dunque possa essere incluso nell'ambito della Continuous Integration.

L'obiettivo di questo lavoro di tesi è infatti stato quello di svincolare completamente i processi di parsing, e quindi anche i processi di rilevamento di code smell, implementati da cASpER rispetto ad IntelliJ IDEA per poi includere l'uso del plugin in un contesto di Continuous Integration. In questo modo è stato reso possibile lanciare il plugin anche al di fuori del suo originario ambiente di esecuzione, con lo scopo di rilevare in maniera automatica i code smell: *Blob*, *Feature Envy*, *Misplaced Class*, *Divergent Change*, *Shotgun Surgery*, *Parallel Inheritance* e *Promiscuous Package*.

Successivamente, implementando la Continuous Integration grazie all'utilizzo del servizio offerto da Travis CI, la nuova versione di cASpER è stata inclusa nella pipeline di test che Travis CI deve eseguire ogni volta che viene effettuata un'operazione di Push verso la repository del progetto che si sta sviluppando.

6.2 Sviluppi Futuri

Durante la descrizione del lavoro svolto si è più volte sottolineato che l'esecuzione da parte della piattaforma Travis CI del plugin cASpER abbia il solo scopo di rilevare e successivamente mostrare, qualora vi fossero, i code smell presenti all'interno del progetto che si vuole analizzare; al contrario però la precedente versione del plugin, disponibile come tool per IntelliJ IDEA, consente anche l'esecuzione automatica delle operazioni di *Refactoring* corrispondenti ai code smell rilevati.

La scelta di evitare le operazioni di refactoring è stata dovuta a due motivazioni. La prima riguarda l'aspetto di sicurezza, in quanto effettuare modifiche al codice tramite servizi esterni risulta piuttosto pericoloso. Infatti Travis CI non consente di norma la modifica di informazioni appena caricate su GitHub. Inoltre anche le operazioni di refactoring sono dipendenti dall'ambiente IntelliJ IDEA, dunque risulta necessario re-implementare completamente tali procedure affinché possano essere eseguite in maniera indipendente da IntelliJ IDEA.

Dunque si pensa di estendere in futuro le funzionalità della nuova versione di cASpER tramite l'aggiunta delle operazioni di *Refactoring*.

Bibliografia

- [1] M. Flower, *Refactoring: improving the design of existing code* , Addison Wesley, 1999.
- [2] F. Palomba, A. Panichella, A. De Lucia, R. Oliveto, A. Zaidman, *A Textual-based Technique for Smell Detection* , In Proceedings of the 24th International Conference on Program Comprehension (ICPC 2016), Austin, USA, 2016, 10 pages, to appear. <https://dibt.unimol.it/staff/fpalomba/documents/C11.pdf>
- [3] N. Moha, Y.G. Guèhèneuc, L. Duchien, and A.F.L. Meur, *DECOR A method for the specification and detection of code and design smells* IEEE Trans. on Software Engineering, vol. 36, no. 1, pp. 20-36, 2010.
- [4] S. Chidamber, C. Kemerer, *A metrics suite for object oriented design*, Software Engineering, IEEE Transactions on, vol. 20, no. 6, pp. 476-493, Jun 1994.
- [5] N. Tsantalis and A. Chatzigeorgiou, *Identification of move method refactoring opportunities*, IEEE Transactions on Software Engineering, vol. 35, no. 3, pp. 347-367, 2009.
- [6] M. Lanza and R. Marinescu, *Object-Oriented Metrics in Practice: Using Software Metrics to Characterize, Evaluate, and Improve the Design of Object-Oriented Systems*. Springer, 2006.
- [7] A. Rao and D. Ram, *Software design versioning using propagation probability matrix*, in Proceedings of Third International Conference on Computer Applications, Yangon, Myanmar, 2005, 2005.
- [8] R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval*. Addison-Wesley, 1999.
- [9] M. F. Porter, *An algorithm for suffix stripping*, Program, vol. 14, no. 3, pp. 130-137, 1980.
- [10] S. Deerwester, S. T. Dumais, G. W. Furnas, T. K. Landauer and R. Harshman, *Indexing by latent semantic analysis*, Journal of the American Society for Information Science, no. 41, pp. 391-407, 1990.

- [11] J. K. Cullum and R. A. Willoughby, Lanczos, *Algorithms for Large Symmetric Eigenvalue Computations* Boston: Birkhauser, 1998, vol. 1, ch. Real rectangular matrices.
- [12] A. Kuhn, S. Ducasse and T. Girba, *Semantic clustering: Identifying topics in source code* Information & Software Tecnology, vol. 49, no. 3, pp. 230-243, 2007.
- [13] D. Poshyvanyk, A. Marcus, R. Ferenc and T. Gyimothy, *Using information retrieval based coupling measures for impact analysis*, Empirical Softw. Engg., vol. 14, no.1, pp. 5-32, 2009.
- [14] A. Marcus and D. Poshyvanyk, *The conceptual cohesion of classes*, in Proceedings of the International Conference on Software Maintenance (ICSM). IEEE, pp. 133-142, 2005.
- [15] Andrea De Lucia, Fabio Palomba, Fabiano Pecorelli, Michele Simone Gambardella, Manuel De Stefano, *cASpER: A Plug-in for Automated Code Smell Detection and Refactoring*, 28 September 2020.
- [16] M. Fowler, K. Beck, J. Brant, W. Opdyke, D. Roberts, *Refactoring: Improving the Design of Existing Code*, Addison-Wesley, 1999.
- [17] Rishi Yadav, Narinder Rana, Harish Kundra, *Continuous Integration*.
- [18] Christopher Blake, SAS, *Improving Mean Time to Resolution and Root-Cause Analysis for Complex SAS® Environments*, 2021.
- [19] Mojtaba Shahin, Muhammad Ali Babar, Liming Zhu, *Continuous Integration, Delivery and Deployment: A Systematic Review on Approaches, Tools, Challenges and Practices*, 2017.
- [20] *Travis CI User Documentation*, <https://docs.travis-ci.com/>.
- [21] Bruce F. Webster, *Pitfalls of Object Oriented Development*, M and T Books, 1st edition, February 1995.
- [22] Arthur J. Riel, *Object-Oriented Design Heuristics*, Addison-Wesley, 1996
- [23] William J. Brown, Raphael C. Malveau, William H. Brown, Hays W. McCormick III, and Thomas J. Mowbray, *Anti Patterns: Refactoring Software, Architectures, and Projects in Crisis*, John Wiley and Sons, 1st edition, March 1998.